

Project Synapse: Org-Wide Zero-ETL Semantic Data Core

ABSOLUTE INTEGRATED MASTER ARCHITECTURE & IMPLEMENTATION BRIEF

0. Abstract & Completeness Target

Project Synapse establishes architectural completeness for an enterprise-wide, discrepancy-tolerant, schema-on-read context integration core. Instead of managing static database migrations, this framework resolves typical runtime data vulnerabilities—including deterministic parsing errors, processing cost runaways, fine-grained multi-tenant isolation, and request latencies—with unified open-source engine blocks and explicit mathematical control primitives.

1. The Problem Landscape: The Schema-on-Write Bottleneck

Enterprise transaction systems naturally emit scattered, unorganized, and highly transactional variations of operational telemetry (OLTP). To derive analytics or reasoning trees from this stream, engineering teams historically build complex **Schema-on-Write** pipelines. This standard design pattern forces data out of active workloads into rigid ETL frameworks (e.g., dbt, Apache Spark) to be aggressively normalized, cleaned, and scrubbed before it enters a warehouse.

Core Systems Failures of Traditional ETL:

- **Brittle Data Contracts:** Upstream operational changes immediately snap downstream pipelines, resulting in production dashboard outages.
- **Latent Value Chains:** Multi-hour batch processing windows isolate operational events from immediate AI interaction.
- **Contextual Destruction:** Aggressive scrubbing eliminates native text anomalies, logs, and behavioral parameters that reasoning engines use to map operational faults.

The Synapse Axiom: Retain raw source payloads permanently inside an immutable landing swamp. Shift modeling rules completely to the query interface via an on-demand Schema-on-Read protocol. Maintain a living, temporal topology mapping entities, assertions, and contradictions as active states, rather than computing a clean database snapshot.

2. Open-Source Component Orchestration

Project Synapse achieves full operational separation by routing raw payloads through four specific engine layers:

1. Graphiti (by Zep)

github.com/getzep/graphiti

System Role: Continuous Context & Temporal Relation Modeler.

Processes raw streams as continuous sequential episodes to autonomously map facts, changing parameters, and temporal relationship shifts without manual database model writing.

Mitigates cross-reference memory gaps; connects separate logs into an evolving graph space automatically.

2. GraphRAG (by Microsoft Research)

github.com/microsoft/graphrag

System Role: Hierarchical Abstractor & Global Query Synthesizer.

Clusters massive narrative sets into layered community descriptions, enabling comprehensive macro analysis where localized vector lookups lose context.

Addresses global systems synthesis; provides structural overviews without fixed data warehousing schemas.

3. Data-Juicer (by Alibaba Tongyi Lab)

github.com/datajuicer/data-juicer

System Role: Multi-Format Ingestion Pipeline & Raw Data Streamliner.

Leverages over 200 sandboxed computing operators to filter, deduplicate, and prepare incoming streams into unified text formats for model ingestion.

Handles byte-level normalization; formats scattered streams at runtime without enforcing rigid destination target schemas.

4. PageIndex (by VectifyAI)

github.com/VectifyAI/PageIndex

System Role: Reasoning-Based Document & Structural Tree Navigator.

Skips high-cost dense vector scaling arrays by rendering layout-aware coordinate paths straight onto raw source files, guiding processing agents directly to targeted text sectors.

Bypasses dense vector database scaling bottlenecks; focuses attention windows directly on precise layout blocks.

3. Control Plane: Integrated Mathematical Specifications

3.1 Information Density Routing Envelopes

To protect query runtime channels from cost escalations or server execution timeouts, the Query Router computes an active Information Density Factor (IDF) over candidate text blocks. The path controller minimizes resource costs (\$V_c\$) and processing latency (\$V_l\$) relative to an absolute confidence target (\$T\$):

$$\text{Minimize: } \alpha V_c + \beta V_l \quad \text{subject to} \quad C_f \geq T$$

The runtime Information Density Factor (IDF) evaluates how tightly relations match relative to the total payload footprint:

$$\text{IDF} = (\text{Count of Evaluated Predicates}) / (\text{Total Token Count})$$

- **High Density (IDF ≥ 0.75):** Bypasses large reasoning systems entirely, executing via fast cross-encoder reranking arrays.
- **Low Density (IDF < 0.25):** Routed through PageIndex visual coordinate trees, isolating token usage to highly targeted text zones.

3.2 Multi-Vector Discrepancy Matrix

Project Synapse maps conflicting records explicitly at query time instead of clean-filtering on write. It evaluates a dynamic Validity Weight (W_v) using Authority Rank (A_r), Lineage Proximity (L_p), and a Temporal Decay coefficient (λ) over elapsed time (Δt):

$$W_v = (A_r \times e^{-\lambda \cdot \Delta t}) + L_p$$

Conflict Typology	Core System Challenge	Mitigation Target
Scalar Clash	Opposing numeric parameters emitted by separate nodes inside the same time frame.	Flags as AMBIGUOUS_CONFLICT; surfaces a split view showing both parameters with lineage links if no human_pin exists.
Temporal Supersession	Downstream watermarks reporting contradictory values over an identical asset object instance.	If the newer event scores a higher W_v , the active view updates state while maintaining the past state as a historic edge.
Domain Overlap	Opposing attribute states claimed across separate domain boundaries (e.g., CRM vs Billing name records).	Resolves via designated authority maps inside the Ontology Registry; links alternative strings as searchable aliases.

3.3 Cryptographic Clearance Tracking

Raw objects travel with a rigid Attribute-Based Access Control (ABAC) block. When summaries are compiled, the control plane calculates a mathematical intersection ($\cap$) of all asset clearances:

$$\text{Derived Fact Key} = \text{ACL}_{\text{Object 1}} \cap \text{ACL}_{\text{Object 2}} \cap \dots \cap \text{ACL}_{\text{Object n}}$$

If the requesting user context fails this validation signature, the node drops entirely from the model attention window before prompt execution.

4. Unabridged Core Schema Matrix

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SynapseDataCore",
  "defs": {
    "RawObject": {
      "type": "object",
      "required": ["object_id", "source_system", "content_hash", "ingested_at", "bytes_ref",
"acl_tags"],
      "properties": {
        "object_id": { "type": "string", "format": "uuid" },
        "source_system": { "type": "string" },
        "content_hash": { "type": "string" },
        "ingested_at": { "type": "string", "format": "date-time" },
        "bytes_ref": { "type": "string", "format": "uri" },
        "acl_tags": { "type": "array", "items": { "type": "string" } }
      }
    },
    "Episode": {
      "type": "object",
      "required": ["episode_id", "raw_object_ids", "domain", "prep_pipeline_version", "acl_tags"],
      "properties": {
        "episode_id": { "type": "string", "format": "uuid" },
        "raw_object_ids": { "type": "array", "items": { "type": "string", "format": "uuid" } },
        "domain": { "type": "string" },
        "time_span": {
          "type": "object",
          "properties": {
            "start": { "type": "string", "format": "date-time" },
            "end": { "type": "string", "format": "date-time" }
          }
        },
        "prep_pipeline_version": { "type": "string" },
        "payload_ref": { "type": "string", "format": "uri" },
        "acl_tags": { "type": "array", "items": { "type": "string" } }
      }
    },
    "Entity": {
      "type": "object",
      "required": ["entity_id", "entity_type", "status"],
      "properties": {
        "entity_id": { "type": "string", "format": "uuid" },
        "entity_type": { "type": "string" },
        "canonical_name": { "type": "string" },
        "aliases": { "type": "array", "items": { "type": "string" } },
        "status": { "type": "string", "enum": ["active", "merged", "deprecated"] }
      }
    },
    "Fact": {
      "type": "object",
      "required": ["fact_id", "subject_entity_id", "predicate", "object", "confidence",
"evidence_refs"],
      "properties": {
        "fact_id": { "type": "string", "format": "uuid" },
        "subject_entity_id": { "type": "string", "format": "uuid" },
        "predicate": { "type": "string" },
        "object": { "type": "string" },

```

```

    "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
    "evidence_refs": { "type": "array", "items": { "type": "string", "format": "uuid" } }
  },
  "Conflict": {
    "type": "object",
    "required": ["conflict_id", "subject_entity_id", "predicate", "competing_fact_ids", "status"],
    "properties": {
      "conflict_id": { "type": "string", "format": "uuid" },
      "subject_entity_id": { "type": "string", "format": "uuid" },
      "predicate": { "type": "string" },
      "competing_fact_ids": { "type": "array", "items": { "type": "string", "format": "uuid" } },
      "status": { "type": "string", "enum": ["open", "resolved", "accepted_plural"] }
    }
  },
  "Claim": {
    "type": "object",
    "required": ["statement", "supporting_fact_ids", "confidence"],
    "properties": {
      "statement": { "type": "string" },
      "supporting_fact_ids": { "type": "array", "items": { "type": "string", "format": "uuid" } },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
    }
  }
}

```

5. Target Validation Track: Operational Infrastructure Incidents

The selected verification path for the sandbox playground targets **Infrastructure & Operational Incidents**[cite: 8]. This track naturally matches our discrepancy model, containing distributed infrastructure alerts that contradict each other continuously during an incident (e.g., CI/CD deploy steps reporting clean container pushes while internal cluster health endpoints declare node loop crashes and developers drop manual rollback messages into live Slack channels)[cite: 8].

6. Playground Roadmap Steps

- **Phase 0:** Sign-off on architecture blueprints, data schemas verification, and validation metrics setup[cite: 8].
- **Phase 1:** Writing a local Python worker acting as a Data-Juicer operator to package raw data logs into compliant RawObject schemas[cite: 8].
- **Phase 2:** Spinning up a Graphiti instance to extract entities and trace conflict states dynamically on read[cite: 8].
- **Phase 3:** Deploying the front-end query application on Vercel hooked into Google AI Studio long-context engines[cite: 8].